软件工程与项目实践 · 作业 2

流程设计
设计规格 · 任务分解 · 编排器
执行可视化与交互

承接《多体讨论工作流设计文档 v1.2》：把已冻结的需求规格翻译为设计规格，拆解为带执行对象与限制时间的任务图，再由确定性编排器调度执行，并全程可视化、可介入。

出品	软件工程实践第四组
文档版本	v1.0
日期	2026-11-12
上游基线	《多体讨论工作流设计文档 v1.2》（作业 1 终版）
配套实现	任务分解 Agent 系统（已实现并实测，见第 8 章）

本文件只处理全链路的后半段：设计规格 → 任务分解 → 编排器 → 执行可视化与交互。想法、多体讨论、建设方案、多方评价与需求规格冻结门的口径，一律以 v1.2 为准，本文不重述、不改写。文中所有量化数字均来自配套系统的真实运行产物，不使用示意数字。

目录

1. 文档定位与承接关系
2. 总体架构
3. 角色与角色卡
4. 设计规格
5. 任务分解（核心章）
6. 硬性记忆与多轮对话
7. 编排器设计
8. 执行可视化与交互
9. 贯穿实例与验证

附录 A　任务卡模板

附录 B　与 v1.2 的继承口径对照表

0 文档定位与承接关系

0.1 本文在全链路中的位置

v1.2 定义的全链路共九个阶段：想法 → 多体讨论 → 建设方案 → 多方评价 → 需求规格（需求确认与冻结门）→ 设计规格 → 开发任务分解 → 开发任务编排 → 执行可视化与控制。前五个阶段已经在 v1.2 中定版，本文接续其后，专门细化最后四个阶段之间的接口与内部规则：设计规格以什么结构进入、任务分解如何产出可直接被调度的任务图、编排器依据什么规则推进状态、人通过什么界面观察和干预执行。

与 v1.2 相比，本文不是改版，而是**向下细化一层**：v1.2 第 6、7、8、9 章分别给出设计规格、任务分解、执行编排与可视化的设计要求；本文把这四章落成可执行的字段、规则、状态机与检查清单，并补上 v1.2 未展开的三件东西——任务分解 Agent 的完整工作方式、硬性记忆与多轮对话的修改通道、以及一组经真实系统验证过的实例数据。

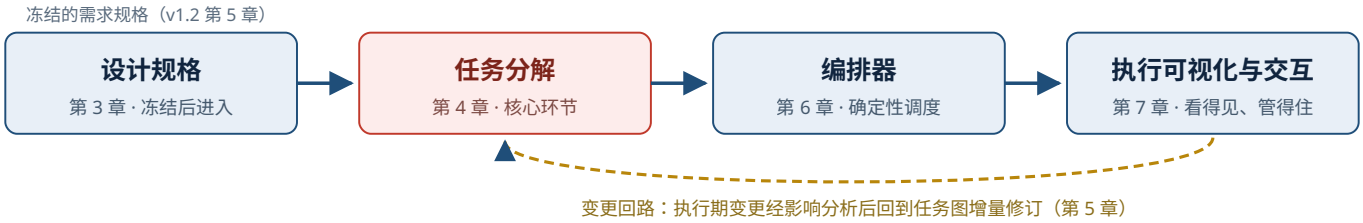


图 0-1　本文覆盖的四段链路与变更回路。任务分解是其中唯一的“内容生产”环节，其余三段以确定性规则为主。

0.2 口径声明（与 v1.2 的关系）

- **唯一基线。**凡 v1.2 已定版的口径——流程确定性、内容概率性；讨论侧“2 个确定性代码组件 + 4 个常驻 LLM Agent”的角色构成；收敛评分与否决项；跨阶段打回上限 2 次；内容哈希追溯；执行安全门；验收标准须“可观察现象 + 阈值或公式”；任务验收方式标注自动／人工／混合——本文全部继承，不另立一套。继承关系逐条对照见附录 B。
- **两处经批准的细化。**其一，v1.2 第 7 章要求任务携带所属模块、需求编号、输入输出、验收方式与预估量级；本文在此基础上把**执行对象**与**限制时间**提升为任务的必填字段与准入门槛（见 4.3）。其二，v1.2 规定任务图是分解与编排之间的正式接口；本文进一步规定：执行对象与限制时间由分解阶段给出正式值并随任务图冻结，编排器按图调度，二者的任何改动都必须走第 5 章的变更提案—确认流程，不允许在执行期口头改派、口头改期。
- **技术路线。**沿用 v1.2 的优先级：流程控制自写状态机优先；AutoGen／CrewAI 类框架只作讨论侧的参照映射，不进入本文四段的流程控制。

0.3 范围与非目标

本文范围内	非目标（明确不做）
设计规格的输入校验、结构定义、追溯与冻结门	多体讨论、评价与需求冻结的规则（v1.2 已定版）
任务分解 Agent 的拆分、依赖推导、分配、限时反推与校验	真实代码的自动生成与部署（执行由执行对象完成，本文只设计调度与控制）
硬性记忆的结构与写改纪律、多轮对话的修改通道	复杂的分层记忆、向量检索与长期记忆学习（明确不需要，见 5.3）
执行编排器的状态机、调度循环、异常与安全门设计	多项目、多租户与组织级权限体系
执行可视化三视图与交互操作设计	LLM 自由拆分任务（LLM 接入为可选占位，默认不启用，见 0.4）

0.4 诚实边界：什么已经实现，什么仍是设计

已实现并实测：任务分解系统——分解引擎（拆分／依赖推导／CPM 关键路径／校验）、角色卡配置、硬性记忆四库、多轮对话（提案—确认—落库）、静态可视化输出（任务清单、DAG 与甘特 HTML）。第 8 章的全部数字即出自该系统的真实运行产物。

本文中的设计（尚未实现为运行时）：执行编排器的完整运行时与交互工作台（第 6、7 章）。其中调度所需的拓扑排序与关键路径计算已在分解系统中实现并经测试任务验证（见 8.5）；状态机的运行时推进、执行适配与工作台交互按本文设计留待后续原型阶段。文中凡涉及编排器行为的表述，均为设计规约，不声称已运行。

> **LLM　　的位置：**分解、分配、估时与校验全部为确定性代码；LLM　　接入仅保留可选占位（如为规格起草工作项草案），默认不启用，且其产出必须走同一套校验才可落库。

1 总体架构

1.1 分层结构

本文四段按“输入越靠上越接近人的判断、越靠下越接近机器的执行”分为五层。层与层之间只通过结构化产物交接，不通过自然语言记录交接：上一层产出什么、下一层消费什么，都写在产物的字段里。



图 1-1　分层架构。硬性记忆与角色卡是贯穿各层的横切设施：前者是全部事实的唯一存储，后者是全部分配决策的依据（第 2、5 章）。

1.2 产物链：四个正式交接物

产物	生产者 → 消费者	核心内容	冻结／版本规则
设计规格（DS）	设计阶段 → 任务分解 Agent	模块清单（含模块级依赖）、接口与数据定义、每模块的需求编号、约束清单引用	带规格版本号与内容哈希；冻结后原文不改，变更走新版本（第 3 章）
任务图（DAG）	任务分解 Agent → 编排器	任务节点（全字段，见 4.4）与依赖边；概要含周期、总工时、关键路径	带图版本号；任何落库修改版本 +1，旧版本保留（第 5 章）
编排实例	编排器按任务图创建	任务图版本引用、各任务运行状态、派发与验收记录	实例只引用某一确定版本的任务图；图中途变更时按 6.6 的规则处理在途任务
执行记录	编排器 → 可视化层与复盘	每次调度、重试、改派、回滚、验收结论，与任务编号绑定	只追加，不删改；与硬性记忆中的变更记录互为索引

产物链的意义在于把“阶段之间的信任”变成“字段之间的校验”：编排器不相信分解者的口头保证，它在装载任务图时逐项校验必填字段；可视化层不相信编排器的内存状态，它只渲染执行记录与任务状态。任何一环发现上游产物不合规，流程停在原地产出错误定位，而不是带病前进。

1.3 两个编排器必须分清

v1.2 第 3 章的主持人／编排器调度的是**讨论中的发言**（谁在第几轮发言、何时收敛），本文第 6 章的执行编排器调度的是**开发任务**（哪个任务何时启动、失败如何处置）。二者同名而异物，混淆会带来真实的工程错误：把发言轮次的逻辑拿来管任务，会丢掉依赖与验收语义；把任务调度的刚性套到讨论上，会扼杀发散。本文的约定：凡称“编排器”且无前缀者，一律指执行编排器；讨论侧的编排只在承接关系处提及，不展开。

1.4 一条贯穿原则：流程确定性、内容概率性

四段之中，凡涉及“流程往哪走”的判断——阶段推进、依赖是否满足、任务能否启动、变更影响哪些任务、限时如何反推——全部由确定性代码与显式规则完成，同一输入必然得到同一输出；LLM 或人的判断只出现在“内容好不好”的层面——规格写得是否完整、评审是否通过、验收是否认可。由此带来两个直接后果：第一，本文的所有关键规则都可以写成检查清单并被程序强制（第 3、4 章的门禁与校验即如此）；第二，任何一次运行结果都可复现、可审计，答辩时能把“为什么这个任务分给了它、为什么截止日是这一天”逐步回放，而不是归因于模型的临场发挥。

2 角色与角色卡

2.1 本段涉及的角色

本文四段中出现的执行角色共五类。其中任务分解 Agent 是规划者，本段不亲自执行任何开发任务；开发、测试、评审三类是执行者与评审者；真人成员承担机器无法裁定的判断与确认。讨论阶段的四个常驻 LLM Agent 属于 v1.2 第 3 章的范畴，其产出经需求冻结后以规格形式进入本文，不在本章重复定义。

2.2 角色卡：统一字段

每个角色一张**角色卡**，字段全组统一。角色卡是硬性记忆的一部分，不是介绍性文档：分解时的分配、对话中的改派复核、编排时的派发，判断依据都是卡上的字段，卡上没有的能力视为不存在。

字段	含义	在流程中的用途
角色名／角色类型	如“开发 Agent／执行者”；类型分规划者、执行者、评审者、真人四类	任务类型与角色类型的准入匹配
定位	该角色在链路中的位置与不承担的边界，一句话	防止职责外溢（如分解 Agent 不下场写代码）
承担者	该角色的具体实例列表（如开发 Agent 下设 A、B 两个实例）	负载均衡的基本单位；增减实例只改此字段
职责	该角色负责事项的清单	评审与复盘时核对“这件事该谁做”
能力标签	可被任务“能力要求”引用的原子标签集合	分配的第一道筛：任务所需能力必须能在卡上找到
负责任务类型	规划／开发／测试／评审／人工中的子集	分配的第二道筛：类型不符直接排除
输入规范／输出规范	接活时应收到什么、交活时必须交付什么	编排器派发与回收时按规范核对
限制与禁止事项	硬性禁区，如“评审者不评审自己参与开发的任务”	分配与改派复核的否决项，命中即拒绝
限时与交付标准	该角色的交付时限口径与逾期处理	与任务限制时间衔接；逾期触发阻塞与升级

完整示例卡：开发 Agent

字段	内容
角色名／类型	开发 Agent ／ 执行者
定位	按设计规格实现各模块的功能代码
承担者	DevAgent-A、DevAgent-B
职责	实现分配到的开发类任务并提交可运行代码；遵守模块接口定义，不私自更改接口；为所写代码提供基本自测说明
能力标签	状态机开发、数据建模、后端开发、前端开发、接口开发
负责任务类型	开发
输入规范	任务单（任务编号、模块与接口定义、输入／输出、验收方式、限制时间）
输出规范	代码与变更说明，注明对应的任务编号与需求编号
限制与禁止事项	不承接测试、评审类任务作为主责；接口需要变更时必须先提出接口变更，由分解 Agent 做影响分析，不得直接改接口后通知
限时与交付标准	按任务限制时间交付；预计超时须提前在对话中提出改限时或改派

其余四张卡同构：任务分解　Agent（规划者，能力含任务分解、依赖分析、估时、调度校验，禁止直接执行开发任务、禁止修改已冻结规格原文）；测试　　Agent（执行者，禁止修改被测代码、禁止降低验收标准迁就进度）；评审角色（评审者，禁止评审自己参与开发的任务、打回必须写明理由）；真人成员（真人，负责领域判断与口径确认，确认结论必须落为可记录的结论并留凭证）。

2.3 分配规则（分解时）

1. **双筛。**先按“负责任务类型包含该任务类型”筛角色卡，再按“能力标签包含该任务的能力要求”筛承担者；两筛都空集时分解报错，指出缺失的能力标签，由人决定是补卡还是改规格，不允许系统擅自放宽。

2. **负载均衡。**在通过双筛的承担者中，选当前累计已分配工时最小的一位。规则刻意保持简单可解释：任何一次分配都能回答“为什么是它”——类型对、能力对、当前最闲。
3. **禁止事项复核。**分配结果逐条比对入选卡的“限制与禁止事项”，命中禁区（如作者与评者同体）即剔除该候选，顺位递补；无合格候选时报错转人工。

2.4 改派规则（运行期）

改派不是换个名字：无论是对话中人工提出，还是编排器因失败触发，目标承担者都必须重过 2.3 的双筛与禁止事项复核，复核不通过当场拒绝并说明缺哪项能力；改派时完整传递任务定义、已产生的中间产物与失败记录（沿用 v1.2 第 8 章的改派要求）；落库后任务图版本 +1 并写入变更记录。一次真实改派的完整过程见 8.6。

2.5 角色卡的维护

增减承担者、增补能力标签都通过修改角色卡配置完成，改后对新分解立即生效，对话改派即时按新卡复核。需要强调一条纪律：**给一张卡加能力标签等于放宽它的承接范围**。例如把“测试”加进开发 Agent 的能力标签，会事实上瓦解“作者与评者分离”一类约束——这类修改应当作为一次显式决策留痕，而不是顺手一改。

3 设计规格

3.1 输入与进入条件

设计规格阶段的输入是经需求冻结门放行的需求规格（v1.2 第 5 章）：需求条目带编号、冻结版本号与确认人凭证。本章的产物——结构化设计规格（DS）——是任务分解 Agent 的唯一合法输入。进入分解前，DS 必须先过 3.4 的冻结门；未冻结的 DS 只能停留在草稿状态，不产出任务图。这样规定是因为分解结果与 DS 版本绑定：DS 若在分解后被悄悄改动，任务图、限时与验收将全部失去依据。

3.2 DS 的结构

承接 v1.2 第 6 章的四部分要求（总体架构、模块划分、接口定义、数据定义），本文把 DS 定为可被程序直接消费的结构，核心是**模块清单**：

结构件	内容要求
规格头	项目名、规格版本号；入库时由系统计算内容哈希并与原文一并保存
模块清单	每个模块含：模块编号（M-x）、名称、一句话说明、所覆盖的需求编号集合、模块级依赖（依赖哪些模块的接口）、模块内的工作项清单
工作项清单	模块内的最小工作粒度（一个工作项将来对应一个任务），每项含：标题、类型（开发／测试／评审／人工）、能力要求、预估工时、验收方式、验收标准
接口与数据定义	模块间接口的输入、输出、错误情形与版本策略；核心数据结构与流向（沿用 v1.2 第 6 章要求，本文不改变其内容规范，只规定其在模块清单中的挂接位置）
约束清单引用	本项目生效的硬约束集合（如关键路径缓冲为 0、打回上限 2 次、安全门前置条件），以编号被任务与检查项引用

把“工作项”写进 DS 而不是留给分解阶段临时发挥，是为了让粒度责任前移：写规格的人必须把每件事写到“可独立验收”的程度（验收标准写不出、工作项就不要放进来），分解 Agent 的工作是把工作项组织成图，而不是替规格作者补粒度。

3.3 模块 ↔ 需求追溯

- **向下覆盖：**每条已冻结需求必须至少被一个模块的“需求编号”集合包含；无落点的需求在冻结门处报错，回到设计阶段补模块或回需求冻结门走变更，不得在设计阶段私自改写需求（继承 v1.2 第 6 章）。
- **向上有据：**每个模块必须至少追溯一条需求；无需求依据的模块不得设立。确属基础设施性质的模块，须在说明中写明设立理由并经确认人签字留痕。
- **向下游传递：**模块的需求编号集合会被其全部任务继承，任务因此天然可回溯到需求——这是第 4 章校验“可追溯性”的数据基础，也是评审任务核对“实现是否逐条对应需求”的依据。

3.4 冻结门（G 门）检查单

DS 进入分解前，由确定性校验加一道人工确认共同把关。检查单全部通过才冻结：

编号	检查项	判定方式	不通过的去向
G-1	结构完整：每个模块字段齐全；每个工作项的标题、类型、能力要求、工时、验收方式、验收标准无一缺失	程序校验	定位到模块／工作项，退回补齐
G-2	需求覆盖：已冻结需求 100% 有模块落点（3.3）	程序校验	列出未覆盖需求编号，退回设计
G-3	接口无孤儿：模块级依赖指向的模块存在；接口定义中声明的提供方与使用方在模块清单中均可找到	程序校验	列出孤儿接口，退回设计
G-4	模块依赖无环：模块级依赖图为 DAG	程序校验	定位成环模块对，转人工裁定拆分方式
G-5	验收可判定：每个工作项的验收标准是“可观察现象 + 阈值或公式”的可判定语句（继承 v1.2 的验收标准形式要求），“完成开发”“效果良好”类表述不予放行	程序初筛 + 人工确认	逐项退回改写
G-6	关键设计决策留痕：每项关键决策注明依据的需求编号与取舍理由（继承 v1.2 第 6 章）	人工确认	补记后重新提交

G-7	基线签署：确认人对冻结版本签字（留凭证），系统记录确认人、时间、版本号与内容哈希	人工确认	停留在草稿状态

3.5 版本与哈希

DS 入库时计算 SHA-256 内容哈希，与原文、版本号、入库时间存入规格库（见 5.2）。同一内容重复入库不产生重复条目（哈希相同即视为同一版本）。哈希的作用不是安全审计的全部，而是回答一个朴素问题：“现在分解所依据的规格，和当时确认的那一份，是不是同一份？”——任何无痕修改都会使哈希对不上，分解与复盘时一查便知。DS 升版后，旧版本保留；基于旧版的任务图不会被自动改写，必须按第 5 章的变更流程显式处理。

4 任务分解（核心章）

4.1 任务分解 Agent 的定位

任务分解 Agent 位于设计规格与执行编排之间，是本文四段中唯一的“内容生产者”：它读入冻结的 DS，产出任务图。它是**规划者而非执行者**——不写任何模块的代码，不替编排器决定运行期调度，只把“有哪些任务、谁来做、什么时候必须做完、彼此什么关系”一次性定清并冻结成图。此后流程的改变不靠重新许愿，靠对任务图的版本化修改（第 5 章）。

4.2 五步流水线

步骤	做什么	产物／规则要点
① 解析规格	读入冻结 DS，展开模块清单、工作项、模块级依赖与需求编号	规格先入规格库（哈希存档）；版本对不上即中止
② 按模块拆分	每个工作项生成一个任务，继承所属模块、需求编号与验收标准	一个工作项 = 一个任务，不合并不再拆（粒度责任在 DS 侧，见 3.2）
③ 推导依赖	按 4.5 的确定性规则在任务之间连边	只连规则能证明的边；推导不出关系的任务保持独立，不臆测
④ 分配与限时	按角色卡分配执行对象（2.3）；按 CPM 反推每个任务的限制时间（4.6）	执行对象与限制时间在此定为正式值
⑤ 校验出图	跑 4.7 校验清单；全过才生成任务图并落库	任一项不过：报错定位、不产出任务图、不落库

五步全部由确定性代码执行。同一份 DS、同一组角色卡、同一个项目截止日，重复分解必然得到同一张任务图——这是本设计敢于把数字写进文档的底气。

4.3 分解硬规则

1. **执行对象必填门槛。**没有执行对象的任务不许存在：分配不到合格承担者的工作项使整个分解失败，而不是生成一个“待指派”任务混进图里。
2. **限制时间必填门槛。**每个任务必须携带完整的限制时间（工时、开始日期、截止日期、缓冲、是否关键路径）。没有期限的任务不许进入编排器——编排器装载任务图时对这两项再做一次准入校验，缺失即拒绝启动。
3. **可独立验收。**一个任务完成后能单独判定通过与否；验收标准来自 DS 的工作项原文，不在分解阶段改写。
4. **可独立交付。**任务产物自成一体，可被下游任务直接使用；产物互相缠绕的两个工作项应在 DS 侧合并或拆清，而不是靠依赖边掩盖。
5. **编号稳定。**任务编号由计数器单调发放、永不复用；任务被删除时编号进入墓碑（记删除时间与原因），后续新任务不占用旧号（见 5.2）。
6. **过大与过小。**沿用 v1.2 第 7 章：大到无法估量或内含多个独立验收点的工作项退回 DS 侧再拆；小到调度成本高于执行成本的工作项应在 DS 侧合并。分解阶段只报告、不擅自增删工作项。

4.4 任务卡全字段

字段	说明	来源
编号	T-xx，全图唯一，稳定不复用	分解时发放
标题	一句话任务名，动词 + 对象	DS 工作项
来源模块／需求编号	所属模块编号与名称；继承的需求编号集合	DS 模块清单
类型／能力要求	开发／测试／评审／人工；所需能力标签	DS 工作项
输入／交付物	开工前应收到的上游产物；完成时必须交付的东西	DS 接口定义与依赖关系
依赖	前置任务编号集合（边的来源见 4.5）	规则推导
执行对象	角色名 + 承担者（如 开发 Agent ／ DevAgent-B）	角色卡分配（2.3）
限制时间	工时、开始日期、截止日期、缓冲小时、是否关键路径	CPM 反推（4.6）
验收方式／验收标准	自动／人工／混合；可判定语句（G-5 已把关）	DS 工作项

标记	如“接口变更待复核”等运行期标记，默认为空	变更流程写入（第 5 章）

空白任务卡模板见附录 A。

4.5 依赖推导规则

依赖只从两种可证明的关系推导，不做语义猜测：

- **同模块流水线：**同一模块内按“人工 → 开发 → 测试 → 评审”的类型顺序成链，后一类任务依赖本模块全部前一类任务。理由是口径先于实现、实现先于验证、验证先于评审；例如测试任务依赖本模块的全部开发任务，而不是其中某一个。
- **跨模块接口依赖：**模块 A 在 DS 中声明依赖模块 B 时，A 的开发任务依赖 B 的开发任务（接口先就绪）；B 没有开发任务时，依赖 B 的全部任务。

两类规则的共同点是可复核：任意一条边都能指回 DS 中的一处声明或一条类型规则。实例中 13 条依赖边的逐边来源见 8.3。

4.6 限时反推：工时制 + 关键路径法（CPM）

限制时间不是逐个任务拍出来的，而是从项目截止日整体反推的，口径为**工时制**（以小时计量、工作日 8 小时／日、跳过周末）：

1. **正推。**自项目起始沿 DAG 计算每个任务的最早开始与最早结束（单位：小时）；全图最长链即关键路径，其长度决定项目所需的最短日历跨度，项目开始日由此自截止日向前数定。
2. **反推。**自项目截止日沿 DAG 逆推每个任务的最晚开始与最晚结束；某任务的最晚结束与最早结束之差即为该任务的**缓冲**。
3. **落日期。**把小时偏移映射到工作日日历，得到每个任务的开始日期与截止日期。

> **关键路径任务的缓冲恒为 0。**这不是排期技巧，而是定义：关键路径上任何一个任务延期多少小时，项目截止日就顺延多少小时。因此关键路径任务在可视化中被单独高亮（第 7 章），在执行中被优先保障资源；非关键路径任务则拥有可被变更消耗的缓冲，变更影响分析会把“吃掉多少缓冲”算给确认人看（5.5）。

选用工时制而非人天制的理由：本组实例中任务工时分布在 4–24 小时，人天粒度（8 小时）会把 4 小时与 8 小时的任务压成同一格，关键路径长度与缓冲都会失真；工时制与程序实现一一对应，数字可复算。

4.7 校验清单（出图前）

编号	校验项	失败动作
V-1	无环：任务依赖图为 DAG	定位成环路径，分解失败
V-2	无悬空：每条依赖指向的任务存在于图中	指出悬空边，分解失败
V-3	必填齐备：每个任务的执行对象与限制时间五项俱全（4.3 的门槛在此执行）	指出缺项任务，分解失败
V-4	可追溯：每个任务至少携带一个需求编号，且该编号属于其来源模块的追溯集合	指出脱链任务，分解失败
V-5	可独立验收：每个任务有验收标准且验收方式在｛自动，人工，混合｝内	退回 DS 侧修订
V-6	分配合规：每个执行对象通过角色卡双筛与禁止事项复核（2.3）	指出违规分配与所犯禁区
V-7	排期自洽：关键路径长度与项目周期相容；关键路径任务缓冲为 0；每个任务的截止日期不晚于项目截止日	报告冲突量，转人工调整截止日或工时

校验的设计取向是“宁可不出图，不出坏图”：任务图一旦进入编排器，所有下游——排期承诺、资源安排、验收安排——都建立在它正确的前提上，错误在分解阶段拦截的成本最低。

5 硬性记忆与多轮对话

5.1 为什么需要“硬性”记忆

项目流程在执行期必然会变：改派、改期、接口变更、上游规格升版。变更要可控，前提是“当前事实是什么”有一个不依赖任何人记忆与模型上下文的答案。本设计把全部事实放进结构化存储**原样保存、精确读取**——不做摘要改写，不靠对话上下文兜底。对话可以忘，事实不能忘；会话结束，事实仍在。这就是“硬性”的含义。

5.2 四库结构

库	存什么	规则
规格库	设计规格原文 + 规格版本 + SHA-256 内容哈希 + 存入时间	相同哈希不重复存；哈希是“规格没被悄悄改过”的凭据（3.5）
任务图库	每个版本的完整任务图 + 墓碑 + 编号计数器	每次落库变更生成新版本，旧版本保留可查；删除任务留墓碑，编号不复用（4.3）
约束清单	带编号（C-xx）的硬约束集合，如必填门槛、关键路径缓冲为 0、打回上限 2 次、安全门前置条件	追加须注明来源；校验与调度按编号引用约束
变更记录	每次变更的序号、时间、类型、说明、来源、涉及的图版本前后	只追加、不删改；没有写进变更记录的修改视为没有发生

写入纪律只有一条但不可破：**先提案、经确认、才落库**。任何修改先生成差异（diff）提案摆到确认人面前，确认后才写入并使任务图版本 +1；未确认的提案在会话结束时自动作废。落库采用整体替换的原子写，避免写一半的中间状态被人读到。

5.3 简单多轮对话模型

修改入口是一个**简单的**多轮对话：只保留本次会话内的对话历史与上下文，使确认人可以连续地下达省略式指令——上一次提到的任务会被记住，于是“详情”“把它改派给 B”“截止再提前两天”这类带指代的说法能被正确解析到具体任务与字段。仅此而已：不建分层记忆，不做向量召回，不让系统“猜”历史事实——凡涉及事实，一律从四库精确读取；会话上下文只负责“你刚才指的是哪个”，不负责“事实是什么”。

5.4 对话可执行的操作

操作	行为	落库前置动作
查看列表／详情	列出全部任务，或给出指定（或指代）任务的全字段	只读，不落库
改派	把任务改派给指定承担者或角色	按角色卡即时复核（2.4），不合规当场拒绝并说明原因
改限时（改工时）	修改任务工时后，按 CPM 重算全图所有任务的限制时间	提案中给出全图受影响的截止与缓冲差异
改限时（改截止）	人工点对点调整单个任务的截止日期，不动其余任务	留调整标记；与全图重算的区别在提案中明示
接口变更	声明某模块接口变化，触发影响分析（5.5）并生成给受影响任务打“待复核”标记的提案	影响集合先展示，确认后才打标落库
确认／取消	落库或丢弃当前提案	确认即版本 +1 并写变更记录；取消不留痕于事实层（提案本身不入四库）
查变更记录	回看最近的变更条目	只读

5.5 变更影响分析方法

影响分析回答的问题是：“这处改动会波及哪些任务、吃掉多少缓冲？”方法固定为图遍历，不凭经验估计：

1. 以变更点（任务或模块）为起点，沿依赖边**向下游**做可达性遍历，得到受影响任务集合；模块级变更先映射为该模块的全部任务，再向下游传递。
2. 对集合内每个任务计算：若按变更重排，其最早／最晚时间如何移动、缓冲还剩多少；关键路径任务在集合内出现时单独标红——它没有缓冲可吃，延期即顺延项目截止日。
3. 分析结果与提案一并呈现给确认人：先看后果，再决定是否落库。确认人拒绝，图保持原版本，执行不受任何影响。

阶段之间的打回沿用 v1.2 的上限：同一产物在相邻阶段间被打回的次数上限为 **2 次**，超限不再自动往返，升级为人工裁决（由真人成员按角色卡的职责处理），裁决结论记入变更记录。

6 编排器设计

> 本章是**设计规约**：编排器的完整运行时尚未实现（见 0.4）。其中调度所需的拓扑排序与关键路径计算已在分解系统中实现并验证；本章定义它未来运行时必须遵守的状态、循环与门禁。

6.1 定位与职责

执行编排器接收某一冻结版本的任务图，把静态的任务结构推进为实际执行过程：判断哪些任务现在可以开始、把任务派发给执行对象、回收并核对验收、处置失败与阻塞、响应人工控制。它不理解任务内容、不评价产出质量——质量由任务自带的验收方式（自动／人工／混合）负责；它只对**状态流转的正确性**负责。这与 1.4 的原则一致：流程归确定性代码，内容归执行对象与验收人。

6.2 任务状态机

状态	进入条件	出口与说明
未就绪	任务图装载完成时的初始状态	全部前置任务到达“已完成”后转就绪
就绪	前置依赖全部完成并通过验收，且通过 6.7 安全门检查	被调度循环派发后转执行中
执行中	调度器派发成功，执行对象已接收任务单	执行对象回报完成，转待验收；回报失败，转失败
待验收	执行对象回报完成	按验收方式判定：通过转已完成；不通过附理由打回，转就绪重做（打回计数 +1，受 5.5 上限约束）
已完成	验收通过	终态；触发下游任务的就绪检查
失败	执行回报失败或执行超时	可重试者回就绪（重试次数受约束清单配置的上限约束）；耗尽转阻塞
阻塞	重试耗尽，或前置任务被跳过导致依赖失效	等待人工处置：改派、改限时、改依赖或跳过；处置后回就绪或转已跳过
已跳过	人工显式跳过（必须填写理由）	终态；其下游任务逐一重新评估依赖是否仍成立

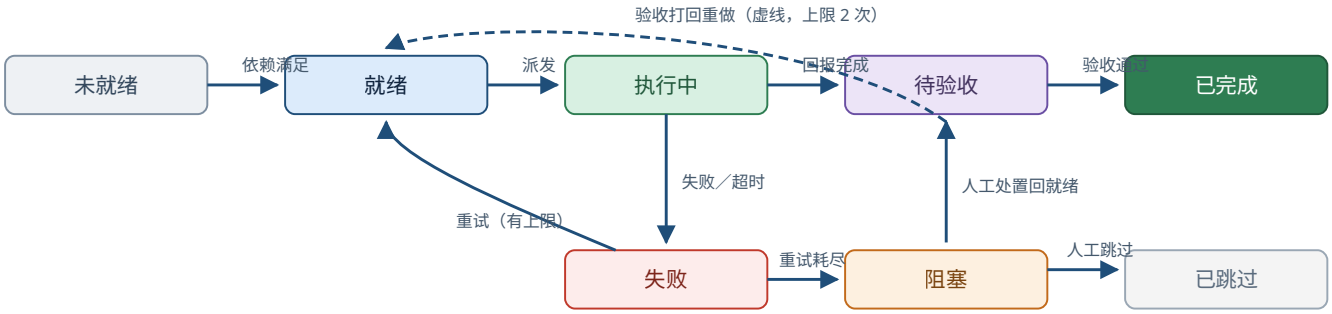


图 6-1 任务级状态机。实例级状态（运行中／等待人工／已完成／已终止）由任务集合的聚合状态推导，人工节点到达时实例进入“等待人工”并暂停派发。

6.3 调度循环

编排器以固定四步循环推进，每一轮的顺序不变：

1. **扫描就绪：**检查全部“未就绪”任务的前置集合，前置全部已完成者进入就绪候选，再逐项过安全门（6.7），合格者转“就绪”。
2. **派发：**在就绪集合中按 6.4 的顺序选取任务，向其执行对象下发任务单（任务编号、输入与上游产物引用、验收方式、限制时间、回报方式）。
3. **回收与验收：**收集执行回报；回报完成者转“待验收”，按验收方式分流——自动验收由测试或脚本当场判定，人工／混合验收使实例进入“等待人工”，在工作台等待验收人结论。验收不通过者附理由打回。
4. **记录与刷新：**本轮全部状态变化写入执行记录并推送可视化层刷新，然后回到第一步。

循环在两种情形下挂起：到达人工节点（实例转“等待人工”），或所有任务均已到达终态。挂起不丢状态：状态全部持久化，任何人工操作或外部事件都能唤醒循环继续推进。

6.4 就绪判定与派发顺序

- **就绪的硬条件：**前置任务全部“已完成”（完成意味着通过验收，而非仅仅执行结束）；安全门通过；任务图版本与实例绑定版本一致。
- **派发顺序：**就绪集合内部先按拓扑层序，再按关键路径优先（缓冲小者先行），最后按任务编号稳定排序——同一状态下重复运行得到同一派发序列。
- **并行与串行：**无依赖关系的任务可并行派发，并行度受资源与成本上限约束（继承 v1.2 第 8 章）。但**同一执行对象默认串行**：分配给同一承担者的任务按图上顺序逐个执行，除非任务卡显式声明可并行承接——因为限制时间的反推以“承担者工时不重叠”为前提，擅自并行会使全部限时失效。

6.5 异常处理矩阵

异常	判定特征	处置
执行失败（可重试）	执行错误、外部依赖抖动等偶发因素	回就绪重试，次数上限由约束清单配置；每次重试写入执行记录
执行失败（不可重试）	输入无效、依赖产物缺失、需要人做决定	直接转阻塞并通知确认人，不做无意义重试
执行超时	超过任务限制时间仍未回报	先查询执行状态；无响应按失败处理，并触发逾期提醒
验收不通过	验收结论为打回且附理由	回就绪重做，中间产物保留供修正参考；打回计数受 2 次上限约束，超限升级人工裁决
上游根因错误	失败根源在设计缺陷或需求偏差，而非本任务执行	不在错误前提下重试；回滚到相应确认门（DS 冻结门或任务图确认门），旧版本保留（继承 v1.2 第 8 章的回滚要求）
结构性错误	装载时发现图有环、悬空或必填缺失	硬停：实例不予创建，错误定位回分解阶段（正常流程下已被 4.7 拦截，此为纵深防御）

6.6 人工节点清单

节点	出现时机	人可做的操作与系统处理
基线确认（DS 冻结门）	设计规格进入分解前	确认／退回补充；确认后冻结并记录确认人、时间、版本与哈希（3.4 的 G-7）
任务图确认	分解出图后、创建编排实例前	整体通过／退回重拆（附结构化理由）／经对话做局部改派、改限时；通过后图版本锁定给实例
人工类任务执行	任务类型为“人工”的任务被派发时	真人成员完成判断类事项并回填结论（留凭证），实例在此等待
人工／混合验收	对应验收方式的任务进入待验收时	逐项核对验收标准，给出通过或打回（附理由）；没有回复不等于同意，实例保持等待
变更确认	改派、改限时、接口变更等提案生成时	先看影响分析，再批准／拒绝／修改提案；批准后落库并按影响集合增量处理在途任务：未开始者按新图，已完成者不受影响，执行中者由人决定继续或中止重派
异常升级裁决	打回或重试超限、阻塞无法自动解除时	由真人成员裁决：接受现状、调整任务图或终止实例，结论记入变更记录

6.7 执行安全门

继承 v1.2 第 8 章：任务进入编排前必须通过安全门——执行 Agent 运行于受限沙箱、采用最小权限、密钥与凭据统一管理不外泄、敏感数据脱敏；安全与合规约束自需求阶段沿可追溯链传递到执行面，作为任务可启动的前置条件。本文在此之上补一条结构性检查：**任务的执行对象、限制时间、验收方式三项在装载时再次校验**，缺失或与角色卡冲突（如承担者已不在卡上）的任务不得启动，实例创建被拒绝并报告缺项。这使 4.3 的必填门槛在执行入口形成第二道防线。

7 执行可视化与交互

> 本章同样是**设计规约**（工作台未实现为运行时，见 0.4）。已实现的是分解产物的静态可视化输出——任务清单与带 DAG、甘特的静态页面，与本章三视图共用同一套数据与着色口径，可视为本章设计的静态形态验证。

7.1 设计原则

- **状态永远诚实：**界面只渲染执行记录与任务状态，不做“看起来完成了”的推测性展示；数据未到就显示未到。
- **操作必定留痕：**每一次人工操作（确认、打回、改派、跳过、回滚）都写入执行记录与变更记录，注明操作者与时间，可事后逐条回放。
- **暂停必须可见：**实例进入“等待人工”时，界面须明确指出当前在等谁的什么决定；没有回复不等于同意，等待状态不设默认通过。

7.2 三视图

视图	呈现什么	回答的问题
DAG 视图	任务图全貌：节点按状态着色（7.3），依赖边方向明确，关键路径整条高亮，同拓扑层的任务视觉成列	结构长什么样、谁卡着谁、关键路径在哪
甘特视图	每个任务的开始—截止条（按限制时间绘制），附项目截止线与今日线；关键路径任务红色，非关键路径任务的条尾以浅色延展表示缓冲	时间上是否还来得及、缓冲还剩多少
清单视图	任务全字段表格，可按模块、执行对象、状态筛选；点任意行打开任务详情（交付物、验收标准、执行日志、变更历史）	某个任务的细节、某类任务的集合状态

7.3 状态着色

未就绪 · 灰　就绪 · 蓝　执行中 · 绿　待验收 · 紫　已完成 · 深绿　失败 · 红　阻塞 · 橙　已跳过 · 浅灰

着色规则在三视图间严格一致，同一状态在任何视图、任何时刻都是同一个颜色；关键路径的强调（节点描边与整链高亮）叠加在状态色之上，不与状态色争夺含义。

7.4 进度口径

项目进度以**关键路径完成工时 ÷ 关键路径总工时**为主口径，全图完成工时比为辅口径并同时展示。理由：非关键路径任务普遍带缓冲，先做完一堆带缓冲的任务会把“完成百分比”刷得很高，而项目最早能结束的时间只由关键路径决定；用关键路径口径报告进度，数字不好看，但不会骗人。任一关键路径任务延期，甘特视图立即以其顺延量重算项目预计完成日并标红提示。

7.5 交互操作与控制面

操作	作用	约束
启动／暂停／恢复	控制整个编排实例的调度循环	暂停只停派发新任务，在途任务到安全点停住；状态全部持久化
重试	对失败任务手动再派发一次	计入重试次数上限；不可重试类失败不提供此入口
改派／改限时	跳入第 5 章的对话变更流程	必须走提案—确认—落库，不在工作台直接改字段
接管	对阻塞任务人工处置（改依赖、跳过、终止实例）	跳过必须填理由；全部操作需二次确认并留痕
回滚	把流程退回指定确认门	仅当失败根因在上游时可用（6.5）；旧版本保留，不覆盖历史
验收处理	对“待验收”的人工／混合任务给出通过或打回	打回必须附理由并计入打回计数；结论留凭证

不可用的操作在界面上直接隐藏而非置灰——置灰会诱发“为什么不能点”的无效沟通，隐藏则让“当前能做什么”一目了然。这条与 7.1 的“暂停必须可见”配合：等待人工的实例，界面只突出当前唯一待办的那一项操作。

8 贯穿实例与验证

8.1 示例规格：多体讨论工作流系统自身

本章用一组真实数据走通全文：示例设计规格就是多体讨论工作流系统自身的设计规格（规格版本 design-v1.0），含 5 个模块、7　　条需求。选它作实例不是图省事，而是因为它同时是被设计的系统与设计的对象——分解结果的每一行都能在 v1.2 与配套实现中找到出处，数字全部可复算。

模块	名称	需求编号	模块依赖	内容概要
M1	讨论引擎	REQ-001、REQ-002	—	确定性状态机驱动六轮讨论协议：轮次推进、发言顺序与终止判断不用 LLM
M2	共享黑板	REQ-003	—	主张表、待澄清清单、候选方案矩阵、分歧日志四部分的读写与冻结快照
M3	任务分解	REQ-004、REQ-005	M2	把设计规格拆为带执行对象与限制时间的任务图（即配套系统自身）
M4	执行编排	REQ-006	M3	按任务图 DAG 调度执行：依赖满足才启动，支持并行、重试、改派与回滚
M5	可视化界面	REQ-007	M3、M4	任务图 DAG 视图、甘特时间轴与控制面（暂停／重试／改派／回滚）

8.2 分解结果总览

以项目截止日 **2026-12-25** 运行分解，得到：

指标	数值
任务总数	12 个（开发 5、测试 4、评审 2、人工 1）
总工时	154 小时
关键路径	T-03 → T-04 → T-06 → T-09 → T-11 → T-12，共 88 小时
项目周期	2026-12-11 → 2026-12-25（工作日口径，8 小时／日；由关键路径长度自截止日反推开始日）
规格存档	规格库条目：版本 design-v1.0，内容哈希 fdfe00f32e14…（SHA-256 前缀）
初始任务图版本	v1（两次对话变更后为 v3，见 8.6）

8.3 任务全表

编号	任务	来源	执行对象	限制时间	依赖	验收（方式：标准摘要）
T-01	实现讨论状态机与六轮协议推进	M1 · REQ-001/002	DevAgent-A	24h · 止 12-15 · 缓冲 54h	—	自动：六轮协议按状态机完整跑通，三类终止条件各触发一次并留记录
T-02	讨论协议与终止条件测试	M1 · REQ-001/002	TestAgent	10h · 止 12-17 · 缓冲 54h	T-01	自动：三类终止情形用例全过，缺陷可复现
T-03	确认黑板四部分的字段口径	M2 · REQ-003	真人成员	4h · 止 12-11 · 缓冲 0h ★	—	人工：字段口径经真人确认并留记录，开发不得自行变更
T-04	实现黑板数据结构与读写	M2 · REQ-003	DevAgent-B	16h · 止 12-15 · 缓冲 0h ★	T-03	自动：四部分可增删改查，冻结快照与内容哈希一致
T-05	黑板读写与快照测试	M2 · REQ-003	TestAgent	8h · 止 12-16 · 缓冲 60h	T-03、T-04	自动：快照冻结后写入被拒，哈希校验通过

T-06	实现任务分解引擎（拆分/分配/估时/校验）	M3 · REQ-004/005	DevAgent-B	20h · 止 12-17 · 缓冲 0h ★	T-04	自动：示例规格可完整拆分，无环/悬空/必填校验全部生效
T-07	关键路径与限时反推测试	M3 · REQ-004/005	TestAgent	8h · 止 12-18 · 缓冲 34h	T-06	自动：手算关键路径与程序结果一致，关键路径缓冲为 0
T-08	分解结果评审（粒度与可追溯性）	M3 · REQ-004/005	Reviewer	6h · 止 12-21 · 缓冲 34h	T-06、T-07	混合：每任务可独立验收且能回溯到需求编号，结论留档
T-09	实现 DAG 调度、重试与改派	M4 · REQ-006	DevAgent-A	24h · 止 12-22 · 缓冲 0h ★	T-06	自动：依赖未满足不启动；失败重试有上限；改派完整传递中间产物
T-10	编排压力与回滚测试	M4 · REQ-006	TestAgent	10h · 止 12-24 · 缓冲 14h	T-09	自动：回滚可回到指定确认门且旧版本保留，并发无死锁
T-11	实现 DAG 与甘特视图及控制面	M5 · REQ-007	DevAgent-B	20h · 止 12-25 · 缓冲 0h ★	T-06、T-09	混合：任务状态与编排器一致，控制操作可达且有二次确认
T-12	界面可用性评审	M5 · REQ-007	Reviewer	4h · 止 12-25 · 缓冲 0h ★	T-11	人工：关键操作路径经评审通过，问题清单清零或留豁免记录

★ 为关键路径任务（底色标示）。全表每行均有执行对象与完整限制时间——4.3 的两道必填门槛在这张真实任务图上的样子，就是如此。

8.4 依赖图

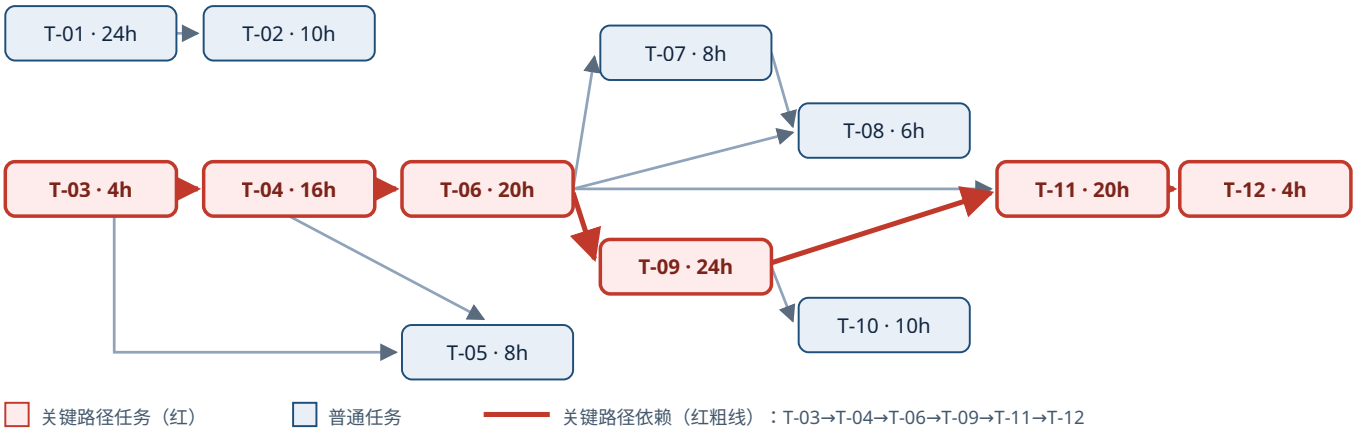


图 8-1　12 个任务的真实依赖图（DAG）。红色节点与红粗线为关键路径（88h）；普通依赖为灰蓝细线。

13 条依赖边全部可按 4.5 的规则回溯来源：同模块流水线边 9 条（M1：T-01→T-02；M2：T-03→T-04、T-03→T-05、T-04→T-05；M3：T-06→T-07、T-06→T-08、T-07→T-08；M4：T-09→T-10；M5：T-11→T-12），跨模块接口边 4 条（T-04→T-06、T-06→T-09、T-06→T-11、T-09→T-11，分别对应 M3 依赖 M2、M4 依赖 M3、M5 依赖 M3 与 M4 的开发任务衔接）。没有一条边来自“感觉上应该先做”。

8.5 排期与负载

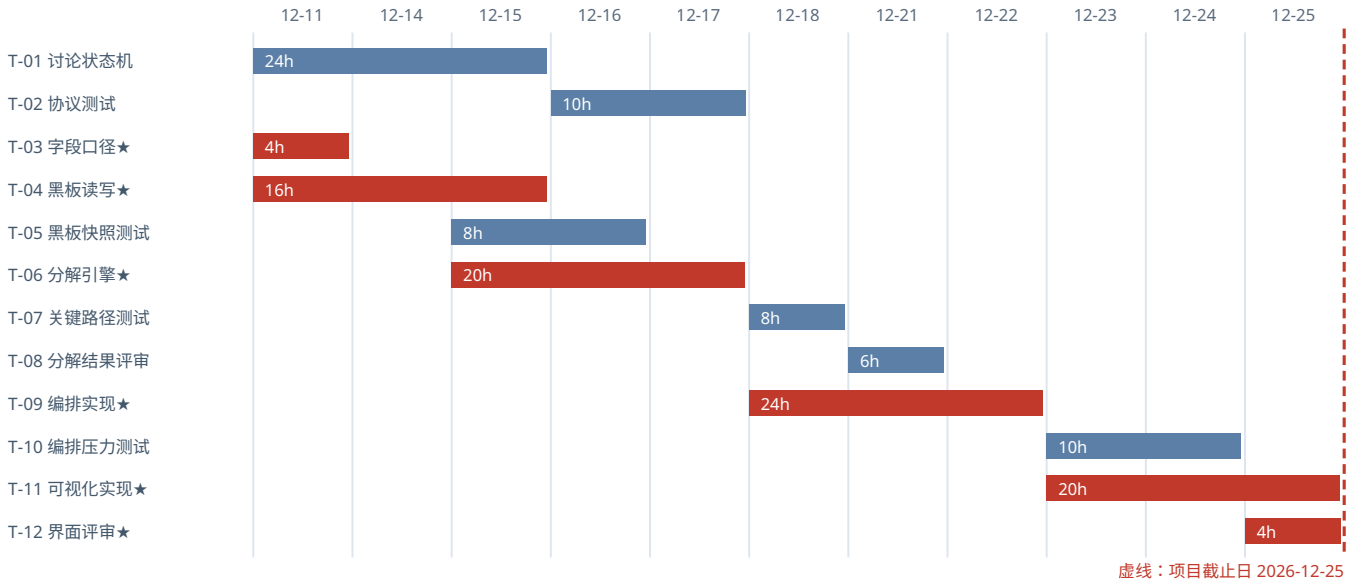


图 8-2　排期甘特（按各任务的开始—截止日期，工作日口径；★与红色条为关键路径任务）。排期是 CPM 反推的结果条，不是逐任务手排：条的右端即 8.3 表中的截止日期。

执行对象（v1 初始分配）	承担任务	累计工时	说明
DevAgent-A（开发）	T-01、T-09	48h	两项均为大工时开发任务；8.6 的改派即发生在此行
DevAgent-B（开发）	T-04、T-06、T-11	56h	三项都在关键路径或其起点上，是全图负载最重者
TestAgent（测试）	T-02、T-05、T-07、T-10	36h	四项测试任务，缓冲普遍较大（14–60h）
Reviewer（评审）	T-08、T-12	10h	含关键路径末端的 T-12
真人成员（人工）	T-03	4h	全图唯一的纯人工任务，且位于关键路径起点：口径确认晚一天，整图顺延一天
合计		**154h**	与 8.2 总工时一致

8.6 两次变更实测

分解出图（v1）之后，用多轮对话做了两次真实变更，均经“提案 → 确认 → 落库”全流程，变更记录可在配套系统的实测日志中逐条查到。

变更一：改派 T-01（任务图 v1 → v2）

```
确认人：改派 T-01 给 DevAgent-B
系 统：改派 T-01 给 DevAgent-B（角色卡复核通过）
将产生以下变化（尚未落库）：T-01 执行对象：DevAgent-A → DevAgent-B
确认人：确认
系 统：已落库：任务图更新为 v2，变更记录已写入硬性记忆。
```

要点：改派先过角色卡复核——DevAgent-B 的能力标签含“状态机开发”且负责类型为开发，复核通过才生成提案；若改派给 TestAgent，会在提案生成前被当场拒绝。落库后 DevAgent-A 的负载降为 24h（仅 T-09），DevAgent-B 升为 80h，这一失衡在清单上一眼可见，正是负载表存在的意义。

变更二：M3 接口变更影响分析（任务图 v2 → v3）

```
确认人：接口变更 M3
系 统：模块 M3 接口变更：
本模块任务（需按新接口调整）：T-06、T-07、T-08
```

```
下游受影响任务（依赖链传递）：T-09、T-10、T-11、T-12
接口变更 M3：给 7 个任务打「待复核」标记
确认人：确认
系 统：已落库：任务图更新为 v3，变更记录已写入硬性记忆。
```

要点：影响集合与 5.5 的方法一致——M3 的本模块任务 3 个（T-06、T-07、T-08），沿依赖边向下游遍历得 T-09、T-10、T-11、T-12，共 7 个任务被打上“待复核”标记；集合内含 4 个关键路径任务（T-06、T-09、T-11、T-12），这一点在提案中意味着：这次接口变更不存在“悄悄改掉”的选项，复核结论出来之前，关键路径上的推进都应视为带风险。未受影响的 5 个任务（T-01～T-05）保持原状，图的其他部分不被无谓地重排。

两次变更后，变更记录共 3 条（含首次分解 1 条），任务图最新版本为 v3；分解时点的交付快照（任务清单、任务图 JSON）仍保留 v1 版本——快照是交付物，记忆库的最新版本才是持续的事实源，两者分工明确。

8.7 实现映射表

本文设计	配套系统中的落点	状态
规格入库与哈希（3.5、5.2）	硬性记忆 · 规格库（SHA-256 内容哈希，相同哈希不重复存）	已实现并实测
冻结门程序校验部分（3.4）	分解引擎的规格解析与校验（字段、验收标准必填）	已实现并实测（人工签署部分为流程约定）
五步流水线与依赖推导（4.2、4.5）	分解引擎：拆分／依赖推导模块	已实现并实测
角色卡分配与改派复核（2.3、2.4）	角色卡配置（5 张）+ 匹配与复核模块	已实现并实测
CPM 限时反推与缓冲（4.6）	分解引擎：关键路径计算；由“关键路径与限时反推测试”用例验证手算一致	已实现并实测
出图校验清单（4.7）	分解引擎的图校验（无环／悬空／必填等）	已实现并实测
硬性记忆四库（5.2）	硬性记忆存储模块（原子写、版本、墓碑、编号计数器）	已实现并实测
多轮对话与变更提案（5.3–5.5）	对话模块：列表／详情（含指代）／改派／改限时／接口变更／确认落库	已实现并实测（8.6）
静态可视化（7.2 的静态形态）	可视化渲染：任务清单 Markdown、DAG + 甘特静态 HTML	已实现并实测
编排器状态机与调度循环（第 6 章）	拓扑排序与关键路径计算已实现并验证；状态机运行时未实现	设计（部分基础已验证）
交互工作台（第 7 章运行时形态）	—	设计，未实现

8.8 验证结论与已知局限

验证结论。（1）分解可复现：同一规格、角色卡与截止日重复运行，任务图逐字段一致；（2）数字自洽：总工时 154h 可由全表逐项加总复核，关键路径 88h 可由链上六个任务工时加总（4+16+20+24+20+4）复核，各执行对象负载加总与总工时一致；（3）变更可审计：两次变更均有提案、确认与落库三段记录，图版本 v1→v2→v3 连续，旧版本可查；（4）门槛真实生效：缺执行对象或缺限时的任务无法出图，改派不合规会被拒绝——这些都不是文档承诺，而是程序行为。

已知局限。（1）工时为规格工作项中的人工预估，系统只做排期计算，不做估时准确性的自动校准；（2）负载均衡采用“累计工时最小”的简单规则，不考虑承担者的熟练度差异与任务的技能匹配深度；（3）依赖推导只覆盖 4.5 的两类可证明关系，语义型依赖（如“内容上参考某任务的结论”）需要人工在 DS 中显式声明为模块依赖，否则不会成边；（4）编排器运行时与交互工作台尚停留在设计阶段（0.4），第 6、7 章的运行时行为未经实测；（5）交付快照与记忆库最新版本并存，使用时需分清“快照看交付、记忆看现状”（8.6 末段）。

附录 A　任务卡模板

空白模板（左列）与一张真实填好的卡（右列，取自 8.3 的 T-06）并置，便于对照字段口径。

字段	模板（填写要求）	示例：T-06
编号	T-xx，系统发放，不手填、不复用	T-06
标题	动词 + 对象，一句话	实现任务分解引擎（拆分/分配/估时/校验）
来源模块	模块编号 + 名称	M3 任务分解
需求编号	继承自模块追溯集合，至少一条	REQ-004、REQ-005
类型	开发／测试／评审／人工	开发
能力要求	须与某张角色卡的能力标签一字不差	后端开发
输入	上游产物引用（编号 + 版本）	T-04 的黑板数据结构与读写接口（随任务图 v1）
交付物	名词短语，一个任务一个主要交付物	可运行的任务分解引擎
依赖	前置任务编号集合，无则写“—”	T-04
执行对象	角色名 + 承担者（必填，缺则不出图）	开发 Agent ／ DevAgent-B
限制时间	工时、开始日期、截止日期、缓冲、是否关键路径（必填，CPM 反推生成）	20h；2026-12-15 → 2026-12-17；缓冲 0h；关键路径：是
验收方式	自动／人工／混合	自动
验收标准	可观察现象 + 阈值或公式的判定语句	示例规格可完整拆分，校验项（无环/悬空/必填）全部生效
标记	运行期标记，默认空	（v3 起）接口变更待复核（模块 M3）

附录 B　与 v1.2 的继承口径对照表

口径	v1.2 出处	本文的落点
流程确定性、内容概率性	第 1 章设计原则	全文总纲（0.2、1.4）；分解、分配、限时、校验全部确定性实现
讨论侧角色构成：2 个代码组件 + 4 个常驻 LLM Agent	第 3 章	仅在承接关系中引用（0.1、2.1），本文不重定义讨论角色；本文的五类角色是开发执行侧角色
全链路九阶段与阶段产物、确认门	第 2 章	本文承接后四段（0.1、1.2），产物链与确认门一脉相承
跨阶段打回上限 2 次	第 2、5 章	变更与打回的上限规则（5.5、6.2、6.5）：超限升级真人裁决
需求冻结、版本与内容哈希追溯	第 5 章	DS 的版本与哈希、规格库（3.5、5.2）
设计规格四部分与覆盖性退出门槛	第 6 章	DS 结构与 G 门检查单（3.2–3.4）
确认人凭证、人工检查点	第 3、5 章	基线签署留凭证（3.4 的 G-7）、人工节点清单（6.6）
WBS 分解、粒度标准、无悬空节点	第 7 章	五步流水线与分解硬规则（4.2、4.3）
任务字段：模块编号、需求编号、输入输出、预估量级	第 7 章	任务卡全字段（4.4）；并按批准的细化新增两个必填字段（下行）
执行对象与限制时间（本文细化）	第 7 章字段的扩展	提升为必填门槛：分解阶段分配与反推出正式值，随任务图冻结，变更走提案—确认（4.3、4.6）
验收方式标注：自动／人工／混合；验收标准形式校验	第 7 章、第 1 章	G-5 门禁（3.4）、任务卡字段（4.4）、编排器验收分流（6.3）
任务图是分解与编排之间的正式接口	第 7 章	产物链（1.2）；编排器装载时二次校验（6.7）
执行编排器能力：依赖调度、并行、重试、改派、回滚、执行记录	第 8 章	调度循环、派发顺序、异常矩阵（6.3–6.5）
执行安全门（沙箱、最小权限、凭据管理、脱敏）	第 8 章	原样继承并补结构性检查（6.7）
两个编排器不可混用	第 3、8 章	专节声明（1.3）
可视化内容与控制操作	第 9 章	三视图、状态着色、进度口径与控制面（第 7 章）
技术路线：自写状态机优先，框架只作参照映射；MVP 最小闭环	第 1、10 章	口径声明（0.2）；配套系统以纯确定性代码实现分解侧最小闭环（第 8 章）

—— 全文完 ——

《流程设计：设计规格 · 任务分解 · 编排器 · 执行可视化与交互》v1.0 · 软件工程实践第四组 · 2026-11-12

配套实现：任务分解 Agent 系统（分解引擎／角色卡／硬性记忆／多轮对话／静态可视化），其实测产物为本文第 8 章的数据来源。